

Accelerated Filtering on Graphs using Lanczos method

Scientific Computing project report

Alberto Defendi

Abstract

Signal processing on graph is becoming . We develop an efficient implementation of the Lanczos method that performs full orthogonalization when loss of orthogonality is detected. We discuss and test ...

Contents

1	Introduction	1
1.1	Signal processing on graphs	1
1.2	The Lanczos method	3
1.3	Algorithm	4
1.3.1	Computational cost	4
1.3.2	Why double orthogonalization?	6
1.3.3	Comparing Lanczos with Arnoldi	6
2	Experiments	8
2.1	Comparing global error with projection error	8
2.2	Performance comparison for Erdős-Rényi graphs	10
2.2.1	Run with selective orthogonalization, $\tau = 10^{-5}$ and $\delta = 10^{-5}$	11
2.2.2	Run with selective orthogonalization, no error bounds and $\delta = 10^{-5}$	12
3	Conclusions & future work	12

1 Introduction

We introduce basic graph theory concepts and briefly overview the results used in the project experiments.

1.1 Signal processing on graphs

We consider a weighted undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W})$, where $\mathcal{V} \subset \mathbb{R}^N$ is the set of vertices, $\mathcal{E} \subset \mathbb{R}^M$ is the set of edges, and $\mathcal{W} : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ is a weight function. The weight function can be represented with a

$N \times N$ matrix W such that $W_{i,j} = \begin{cases} W(v_i, v_j), & \text{if } (v_i, v_j) \in \mathcal{E} \\ 0, & \text{otherwise} \end{cases}$ for all $i, j = 1, \dots, |\mathcal{V}|$, and for our needs, we assume W to be symmetric, that is $W_{i,j} = W_{j,i}$.

Remove
one leg-
end, make
tighter

In the study of signal processing on graph, we model the idea of sending a signal to a node as assigning a value to a vertex with a function $s : \mathcal{V} \rightarrow \mathbb{R}$, whose values can be represented as a vector $s = s(\mathcal{V}) = [s_1, \dots, s_N] \in \mathbb{R}^N$ where each entry $s_i := s(v_i)$ represents the signal sent over a node $v_i \in \mathcal{V}$. We also keep track of the weight of each vertex with a function $d(i) := \sum_{j=1}^N W_{i,j}$, that we represent with the matrix $D = \text{diag}(d(1), \dots, d(N))$.

In this setting, we introduce the graph Laplacian $\mathcal{L}$ defined as $\mathcal{L} = D - W$, where D is the diagonal degree matrix with entries $D_{ii} = d(i)$. By construction, $\mathcal{L}^T = \mathcal{L}$, thus the Lanczos method can be safely applied to our problem.

Remark 1.1. The Laplacian $\mathcal{L}$ is symmetric and semi-definite (it is diagonally dominant), hence by the spectral theorem it admits the decomposition

$$\mathcal{L} = U \Lambda U^*,$$

where $U = [u_0, \dots, u_{N-1}] \in O(N)$ is called Fourier basis, and $\Lambda = \text{diag}(\lambda_0, \dots, \lambda_{N-1})$ is the matrix of eigenvalues of $\mathcal{L}$. Without loss of generality we can assume $0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{N-1}$ and the vectors in U to be in the same order that the eigenvalues.

Definition 1.2 (Graph signal). A graph signal is a continuous function $g : \mathbb{R}^+ \rightarrow \mathbb{R}$. For our needs, we will assume g to be defined on the spectrum of $\mathcal{L}$.

Diagonalising the Laplacian would give an easy way to compute the function $g(\mathcal{L})$ by evaluating it on the eigenvalues of $\mathcal{L}$, which means computing $g(\mathcal{L}) = U g(\Lambda) U^*$. In matrix notation, applying a filter to a graph $\mathcal{G}$ corresponds to the operation

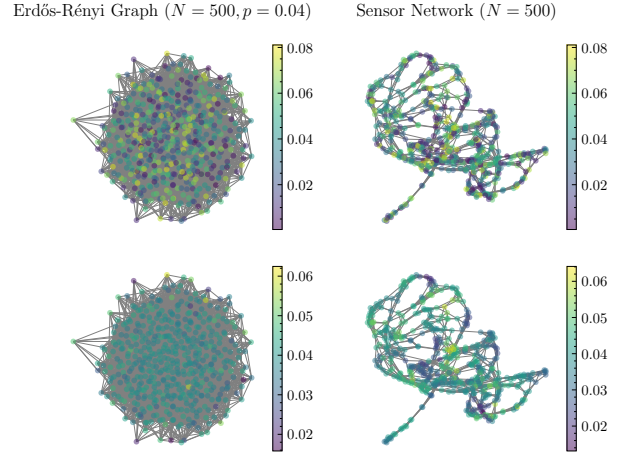
$$s' := g(\mathcal{L})s = U g(\mathcal{L}) U^* s.$$

Remark 1.3 (Computational cost). Computing the Fourier basis for $\mathcal{L}$ is computationally expensive for large graphs, because we diagonalizing the Laplacian with a general method. In Section 1.3.1 we'll see that Lanczos method for $g(\mathcal{L})$ offers an efficient alternative for computing $g(\mathcal{L})$.

In this work, we will restrict ourselves to two graph models: Erdős-Rényi and Sensor graphs. We shall define them as the current implementation in Library [1]

Definition 1.4 (Erdős-Rényi graph). We define the Erdős-Rényi graph $\mathcal{G}_{N,p}$ with N nodes, and probability

Figure 1: Filtering signal on graphs.



p , as the graph constructed by randomly connecting two nodes independently from any other edge, with probability p .

Definition 1.5 (Sensor graph). A sensor graph $\mathcal{G}_N$ is the graph constructed by picking N points on the square $[0, 1] \times [0, 1]$ and connecting each point with edges using k nearest neighbours.

Remark 1.6. As currently stating in PyGSP sensor graph source code, definition of Sensor graph changed in 2019, switching to model generation using k-means algorithm. This might give slightly different results than the 2015's article [2] we are trying to reproduce, but we don't investigate this difference.

Proposition 1.7. *The Laplacian matrix $\mathcal{L} \in \mathbb{R}^{N \times N}$ satisfies the following properties:*

1. $\mathcal{L}$ is symmetric and positive semi-definite.
2. The smallest eigenvalue of $\mathcal{L}$ is 0 with corresponding eigenvector the constant one vector $\mathbf{1}$.
3. $\mathcal{L}$ has N non-negative real-valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.

Proof. See [3]. □

1.2 The Lanczos method

Definition 1.8 (Krylov subspace). Given a matrix $A \in \mathbb{R}^{N \times N}$ and a vector $b \in \mathbb{R}^N$, the Krylov subspace of order j is defined as the set $\mathcal{K}_j(A, b) = \text{Span}(b, Ab, A^2b, \dots, A^{j-1}b)$. We represent the basis of this subspace with a matrix $V_M = [v_1, \dots, v_M] \in \mathbb{R}^{N \times M}$.

From the Arnoldi relation

$$AV_j = V_j H_j + h_{j+1,j} v_{j+1} e_j^*, \quad H_j = V_j^* A V_j = \begin{bmatrix} h_{11} & \dots & \dots & h_{1,j} \\ h_{21} & h_{22} & & \vdots \\ & \ddots & \ddots & \vdots \\ & & h_{j,j-1} & h_{j,j} \end{bmatrix},$$

we can derive the Lanczos method for a symmetric, positively defined matrix A . Setting

$$\alpha_j := h_{j,j}, \quad \beta_j := h_{j+1,j}.$$

From our hypothesis on A , it follows that the matrix H_j , which is both upper and lower Hessenberg matrix, is also symmetric; in other words, it is tridiagonal—thus we rename it T_j . Hence the Arnoldi relation takes the form:

$$AV_j = V_j \alpha_j + \beta_j v_{j+1} e_j^T, \quad T_j = V_j^T A V_j = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \ddots & \ddots & \\ & \ddots & \ddots & \beta_{j-1} \\ & & \beta_{j-1} & \alpha_j \end{bmatrix}.$$

Proposition 1.9. Consider the filter $g : [0, \lambda_{\max}] \rightarrow \mathbb{R}$ and a signal vector $s \in \mathbb{R}^N$. Then the Lanczos method approximates $g(\mathcal{L})s$ with:¹

$$g(\mathcal{L})s \approx V_M \|s\|_2 g(T_M) e_1 =: g_M. \quad (1)$$

Numerically, we compute the eigendecomposition (Λ, U) from T , and then project $g(T_M) = U g(\Lambda) U^\top$ using the equation above.

1.3 Algorithm

In floating point arithmetic the basis V suffers from loss of orthogonality, and orthogonalizing the incoming vector w only against v_j, v_{j-1} becomes insufficient: it comes the need to reorthogonalize the new vector w against all the previous basis vectors. To do so, we implement Algorithm 1, following [2] and [4] with these criteria: full reorthogonalization is triggered if loss of orthogonality is detected after calling Lanczos without orthogonalization, that is when V_M is weakly orthogonal, that is $\|V_M^\top V_M - I\|_2 > \delta$ for a chosen threshold $\delta = 10^{-10}$. When the algorithm is called passing a given threshold τ , it computes the residual $\|r_j\|_2 = \|g_j - g_{j-3}\|$, which computes g using Equation 1, recalling that $g_M := \|b\|_2 V_M g(T_M) e_1$, where $g(T_M)$ is computed by diagonalizing T_M : because T_M is symmetric, banded tridiagonal, computing its eigendecomposition is stable and efficient; indeed the structure of T_M allows us to use a divide and conquer algorithm, and we'll use `eigh_tridiagonal` routine from Scipy.

Check
threshold

Remark 1.10. From now on we'll sometimes refer to Lanczos with double/full orthogonalization as full Lanczos and as Arnoldi with double orthogonalization as Arnoldi.

1.3.1 Computational cost

Here we overview computational cost of the algorithm that we tested, for an input matrix A , where $\text{nnz}(A)$ is the number of non-zero elements of A :

- **Lanczos:** Computes V with partial orthogonalization, with $O(M \cdot N + M \cdot \text{nnz}(A))$ for a sparse matrix². When we need to evaluate function g at each iteration, such as in Section 2.2, we must consider also the $O(N^2)$ operations to compute $g(\mathcal{L})$ with divide and conquer, thus the total cost in this case becomes $O(M \cdot N^2)$.
- **Lanczos algorithm with double orthogonalization:**, computing $w = w - \sum_{i=1}^j v_i (v_i^\top w)$ can cost up to $O(M \cdot N)$, thus the total cost becomes $O(M^2 N + M \cdot \text{nnz}(A))$, and when we also evaluate g it becomes $O(M^2 N + M N^2)$.
- **Arnoldi with double orthogonalization:** For the orthogonalization it takes $O(M^2 N)$, plus the matrix vector product at each step with A , hence again $O(M^2 N + M \cdot \text{nnz}(A))$. Cost is higher when

¹This results holds in general for any function $f : \Omega \rightarrow \mathbb{C}$, where $\Omega \subset \Lambda(A)$.

²note that the product Av_j dominates the cost, and that $\text{nnz}(A) = N^2$ when A is dense.

Algorithm 1 Modified Lanczos method in floating point arithmetic.

Input: Symmetric matrix $A \in \mathbb{R}^{N \times N}$, vector $b \in \mathbb{R}^N$, $M \in \mathbb{N}$, and g function.
Output: Basis matrix $V_l = [v_1, \dots, v_l] \in \mathbb{R}^{N \times l}$ for $K_l(A, b)$, vectors $\alpha = \{\alpha_1, \dots, \alpha_l\} \in \mathbb{R}^l$, $\beta = [\beta_1, \dots, \beta_{l-1}] \in \mathbb{R}^{l-1}$, where $l \in \{1, \dots, M\}$ is the stopping iteration index.
 Initialise $V \leftarrow \mathbf{0}^{N \times M}$, $\alpha \leftarrow \mathbf{0}^N$, and $\beta \leftarrow \mathbf{0}^{N-1}$
for $j = 1, \dots, M - 1$ **do**
 $w \leftarrow Av_j$
 $\alpha_j \leftarrow v_j^\top w$
 $\left\{ \begin{array}{ll} w \leftarrow w - \alpha_j v_j & \text{No reorthogonalization} \\ \text{if } j > 0 \text{ then } w \leftarrow w - \beta_j v_{j-1} \text{ end if} & \end{array} \right.$
 $\left\{ \begin{array}{ll} w = w - \sum_{i=1}^j (w^\top v_i) v_i & \text{Full reorthogonalization} \\ w = w - \sum_{i=1}^j (w^\top v_i) v_i & \end{array} \right.$
 if $j < M$ **then**
 $\beta_j = \|w\|_2$
 if $\beta_j = 0$ **then**
 return $V_j, \alpha_{1:j}, \beta_{1:j-1}$ ▷ — Breakdown —
 else
 if $\|g_j - g_{j-3}\|_2 < \tau$ **then**
 return V_j, α, β ▷ — Breakdown —
 end if
 $v_{j+1} = \frac{w}{\beta_j}$
 end if
 end if
end for
return $V_M, \alpha_{1:M}, \beta_{1:M-1}$.

Algorithm 2 Approximation of $g(\mathcal{L})$ s using Lanczos.

Input: Function g , Matrix T , signal s .
 $\lambda, U \leftarrow \text{diagonalize}(T)$ ▷ Here λ is the vector of eigenvalues.
 $u_1 \leftarrow (U^\top)_{:,1}$
 $y \leftarrow \|s\| U (\text{Diag}(g(\lambda)) U^\top) e_1 = \|s\| U (g(\lambda) \odot u_1)$
▷ Here $u \odot v$ denotes the element-wise product between two vectors u and v .
return Vy

Algorithm 3 Evaluation of $g(\mathcal{L})$ s using Fourier basis.

Input: Function g , Graph $\mathcal{G}$, signal s .
 $\Lambda, U \leftarrow \text{Fourier_basis}(\mathcal{G})$
return $Ug(\Lambda)U^\top s$

evaluating g , due to the fact that matrix H is not perfectly symmetric and tridiagonal (it is up to a small epsilon). Thus we must use standard QR for computing the eigendecomposition, which adds up to a brutal $O(M \cdot N^3)$.

Algorithm 4 Orthogonalization selection.

```
 $V, T, j \leftarrow \text{Lanczos}(A, b, \text{classic})$   
if  $V^T V - I < \delta$  then  
     $V, T, j \leftarrow \text{Lanczos}(A, b, \text{full})$   
end if
```

We observe that in Lanczos computing g it not necessary a drawback when A is dense or when few iterations are performed. It should also be noted that these algorithms suffer from a significant $O(M \cdot N)$ memory cost for storing V_M .

1.3.2 Why double orthogonalization?

From the experimental setting in Section 2.1, we observed that the orthogonality of the basis V with Arnoldi was worse than Lanczos basis. Double orthogonalization showed good results. One reason could be that the Laplacian matrices that we are considering are badly conditioned. The following example motivates this choice:

Example 1.11. We consider a Erdős-Rényi graph with $N = 1000$, $M = 200$, $p = 0.04$. The associated Laplacian $\mathcal{L}$ is terribly conditioned, indeed $k_2(\mathcal{L}) = 2.365029616834158 \times 10^{17}$:

Table 1: Correlation between orthogonalization type and basis orthogonality for Lanczos.

Orthogonalization type	$\ V^T V - I\ _2$
Partial	$6.7883747792312356 \times 10^{-15}$
Single Gram-Schmidt orthogonalization	$1.7133096895857787 \times 10^2$
Double Gram-Schmidt orthogonalization	$6.7883747792312356 \times 10^{-15}$

As we can observe in Table 1, orthogonalizing two times against the previous basis vectors yields to a better orthogonal basis V , while a single Gram-Schmidt iteration performs worse than standard Lanczos.

1.3.3 Comparing Lanczos with Arnoldi

Here we do a short numerical comparison of Lanczos and Arnoldi with double orthogonalization, which should reflect the cost analysis from Section 1.3.1. Algorithm 5 reports our implementation of Arnoldi.

We consider again function the function g used in the experiments and $N \in \{300, 400, 500, \dots, 3000\}$, $M = 200$ and $p = 0.04$. In Figure 2 Lanczos outperforms Arnoldi, but curves follow a similar behaviour.

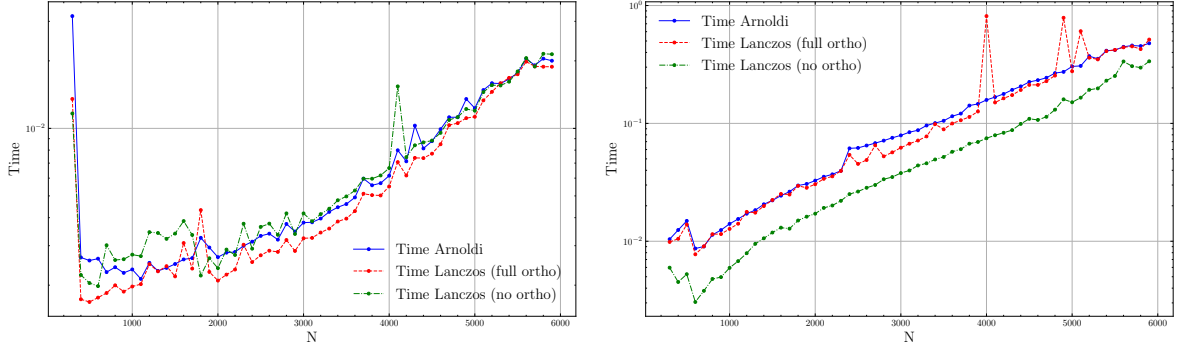
In our implementation we tested that the matrix H returned by Arnoldi is tridiagonal (up to some small perturbations outside the diagonals) and symmetric (up to a small epsilon).

Algorithm 5 Arnoldi with double orthogonalization.

```
1: Input: Matrix  $A$ , starting vector  $s$ , maximum dimension  $M$ , function  $g$ , tolerance  $\tau$ .
2:  $v_1 \leftarrow s / \|s\|_2$ 
3: Initialise  $V \leftarrow \mathbf{0}^{N \times (M+1)}$  and  $H \leftarrow \mathbf{0}^{(M+1) \times M}$ 
4:  $V_{:,1} \leftarrow v_1$ 
5: for  $j = 1 \dots M$  do
6:    $w_1 \leftarrow Av_j$ 
7:   ▷ First orthogonalization
8:    $h_1 \leftarrow V_j^\top w_1$ 
9:    $w_1 \leftarrow w_1 - V_j \cdot h_1$ 
10:   $v_{j+1} \leftarrow w_1 / \|w_1\|_2$ 
11:  ▷ Second orthogonalization pass
12:   $h_2 \leftarrow V_j^\top v_{j+1}$ 
13:   $w_2 \leftarrow v_{j+1} - V_j \cdot h_2$ 
14:   $H_{1:j,j} \leftarrow h_1 + h_2$ 
15:   $H_{j+1,j} \leftarrow \|w_1\|_2 \|w_2\|_2$ 
16:
17:  if  $H_{j+1,j} = 0$  then
18:    break ▷ Arnoldi breakdown
19:  end if
20:  ▷ Convergence check
21:  if  $j > 3$  and  $j < M$  then
22:    if  $\|g_j - g_{j-3}\|_2 < \tau$  then
23:      return  $V_j, H_{1:j,1:j}, j$ 
24:    end if
25:  end if
26:   $v_{j+1} \leftarrow w_2 / \|w_2\|_2$ 
27:   $V_{:,j+1} \leftarrow v_{j+1}$ 
28: end for
29: Set elements of  $H$  smaller than machine epsilon to 0
30: return  $V_{:,1:j}, H_{1:j,1:j}, j$ 
```

Table 2: Mean and variance of values in Figure 2.

	With τ bound		Run until M	
	Mean	Variance	Mean	Variance
Time Lanczos (no ortho)	0.007247	0.000032	0.077766	0.008187
Time Lanczos (full ortho)	0.006398	0.000030	0.159727	0.038419
Time Arnoldi	0.007336	0.000042	0.141613	0.020193
Stopping index Lanczos (no ortho)	11.526316	25.396617	200.000000	0.000000
Stopping index Lanczos (full ortho)	11.771930	46.572055	200.000000	0.000000
Stopping index Arnoldi	11.771930	46.572055	200.000000	0.000000



(a) The algorithms stop when the threshold $\tau = 10^{-6}$ is met. (b) Plot run until M is reached. We see that in most cases they match the complexity analysis.

Figure 2: Comparing Lanczos and Arnoldi. When stopped with threshold we see that theoretical complexity not always match—indeed here full orthogonalization is converging faster than the standard one, probably due the fact that the spectrum of T_l falls more outside the definition interval of g .

2 Experiments

In the following section we generate random Erdős-Rényi and Sensor graphs, as defined in Definitions 1.4 and 1.5, to study Lanczos algorithm's accuracy and performance. We begin by defining some quantities that will be useful in the experiments:

Definition 2.1. (Errors) We define the Lanczos *iteration error* as $\|g_{M+j} - g_M\|_2$, where j is small, and the *true error* as $\|e_M\|_2 = \|g(\mathcal{L})s - g_M\|_2$.

We also define error bounds:

Definition 2.2. We indicate with $\tau =: \tau_{\text{STOP}}$ the algorithm stopping threshold, that is when $\|g_{M+3} - g_M\|_2 < \tau$, and with $\delta =: \delta_{\text{ORTH}}$ the orthogonality threshold, that is when $\|V^\top V - I\|_2 < \delta$.

2.1 Comparing global error with projection error

Scope of this experiment is to numerically verify the approximation in Equation (1). We will do it by relating the following residuals, which represent the algorithmic error and the ground truth $g(\mathcal{L})$. Let's recall some quantities that will be useful in this section: N is the number of graph nodes and size of the Laplacian $\mathcal{L}$, M is the maximum size of the Krylov subspace, and p the probability of the graph.

Experiment. We set $(N, M, p) = (500, 200, 0.04)$ and we try to replicate the Experiment 1 in Article [2].

Recall that computing $g(\mathcal{L})s$ is done by computing the Fourier basis for the graph $\mathcal{G}$, which feasible for the small graph we are considering.

As filter-bank, we consider the function $g(t) = \sin(\frac{1}{2}\pi \cos^2(\pi t))\chi_{[-\frac{1}{2}, \frac{1}{2}]}$. We adapt this function to the

Definition?

graph spectrum, using the `Itersine` class in `Pygsp`, which we plot in Figure 3. Sadly, the authors in the paper apply another transformation to this function called *filter warp*, which we weren't able to reproduce as it was not documented, both in the article and library. Using the algorithm described in Section 1.3, and computing the errors for $j = 1 \dots M$, we obtain the plot in Figure 4:

Figure 3: Itersine plot.

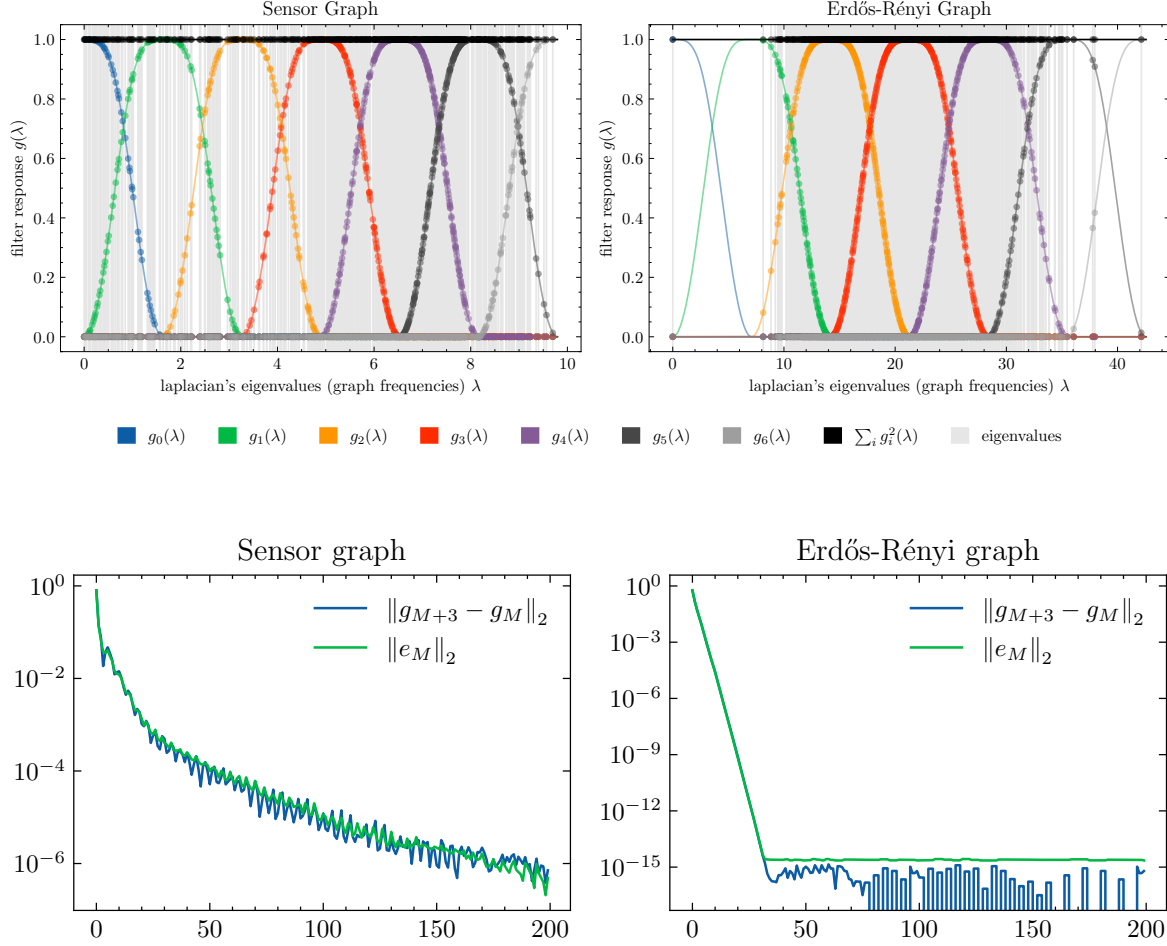


Figure 4: Comparing iteration error with true error.

Let's analyse Figure 4: we immediately observe that the error curves are closely tight, and that their value is decreasing, which shows that the approximation of g gains precision as the basis V gets larger. We also see how the eigenvalues of $\mathcal{L}$ and T influence the steepness and smoothness of the curves. Indeed, when eigenvalues fall outside $[-\frac{1}{2}, \frac{1}{2}]$, we see that the curves rapidly reach machine precision: this is unfortunately caused by the itersine not perfectly matching the Laplacian eigenvalues distribution. We also observe evident lack of smoothness in the iteration error curve. We believe that due the irregularity of $g(t)$ in $t \in \{-\frac{1}{2}, \frac{1}{2}\}$, the eigenvalues of T tend to jump in and out the interval $[-\frac{1}{2}, \frac{1}{2}]$, causing spikes.

I'm not really sure about this.

2.2 Performance comparison for Erdős-Rényi graphs

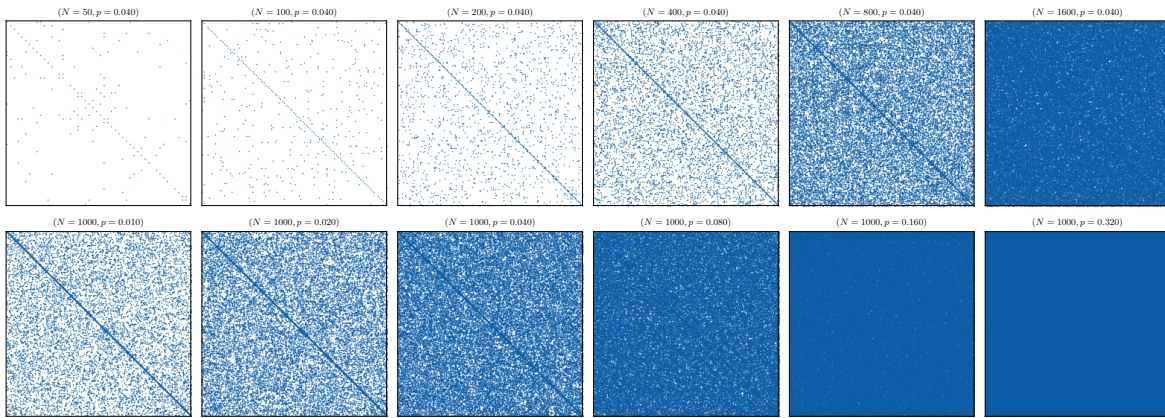
In this experiment, we will study the performance correlation for running Lanczos on Erdős-Rényi graphs $\mathcal{G}(N, p)$, comparing time data when only N varies and when only p varies. In these graphs, N and p dictate the expected properties of the matrix $\mathcal{L}$, that is dimension and sparsity, respectively. In fact, when N increases, we get expect higher computational cost due to the increased cost of the matrix-vector multiplication $\mathcal{L}v$, but when p increases, $\mathcal{L}$ becomes more dense, requiring more floating point operations for the algorithm and so increasing runtime. This patten can be seen from Figure 5, which plots some matrices we used. We choose $\delta = 10^{-5}$, as we experimentally observed that smaller thresholds rarely triggered full orthogonalization. As described in Algorithm 1 our stopping criteria happens when $\|g_j - g_{j-3}\|_2 < \tau$, at the step j .³

Experiment. In the following experiment, we choose generate two sample datasets, letting $n = 6$:

1. N varies in $\{50, 100, \dots, 2^n \cdot 50\}, p = 0.04$.
2. p varies in $\{0.01, 0.02, \dots, 2^n \cdot 0.01\}, N = 1000$.

We repeat the generation of these graphs 200 times to avoid graph randomness biasing our results.

Figure 5: Plot of the sparsity patten of the generated matrices, as N, p varies.



Redo plot
with larger
(N,p)

For time telemetry, we adopt Python's `time.perf_monitor()` which ensures precise measurement. When measuring time, first the algorithm runs without orthogonalization, and if full orthogonalization is triggered, it is taken in the total measurement. Table 3 shows that in some cases it is convenient to have multiple ways to orthogonalize. From these first examples, we can see that we need full orthogonalization when the matrix is either too sparse or too dense.

We now show plot of some runs of this experiment for some conditions on τ, δ and the stopping criteria.

³Here index goes backwards because matrix T_l would not be defined for $l > j$ at iteration j !

2.2.1 Run with selective orthogonalization, $\tau = 10^{-5}$ and $\delta = 10^{-5}$

We observe that when N or p increase the computational time decreases. In the first case, since N becomes larger than M , from the theory we know Krylov subspace is approximated better by the algorithm when $N \gg M$: this reflects in less iterations for the algorithm, as we see in Table ?. In the second case, we expect from Figure 5 that performances should increase with p because matrices are getting densier. Instead, with small p , matrices are sparse, but iteration count and time is high, meaning that full orthogonalization is usually performed, while when p is high, the algorithm runs few operations in average and time is fast. We believe that this has to do with the phenomena of eigenvalue distribution we already observed in Section 2.1, thus we are again experiencing subsequent iterations error reaching machine precision fast as depicted in Figure 4. Unfortunately, we cannot fix this behaviour but we can try running. Unfortunately, we cannot fix this behaviour but we can try running. Unfortunately, we cannot fix this behaviour but we can try running the process excluding the bound arrest.

Table 3: Lanczos execution tests when N varies and p fixed (top), and vice versa (bottom).

N	M	p	Time (s)	Orthogonalization	Breakdown	Stopping index	ε
50	200	0.040000	0.042984	Full	Bound	39	0.000010
100	200	0.040000	0.096729	Full	Bound	73	0.000010
200	200	0.040000	0.165156	Full	Bound	87	0.000010
400	200	0.040000	0.001939	Partial	Bound	15	0.000010
800	200	0.040000	0.002347	Partial	Bound	13	0.000010
1600	200	0.040000	0.003858	Partial	Bound	11	0.000010
1000	200	0.010000	0.153377	Full	Bound	122	0.000010
1000	200	0.020000	0.002103	Partial	Bound	15	0.000010
1000	200	0.040000	0.002052	Partial	Bound	13	0.000010
1000	200	0.080000	0.001625	Partial	Bound	10	0.000010
1000	200	0.160000	0.003931	Full	Bound	9	0.000010
1000	200	0.320000	0.005302	Full	Bound	8	0.000010

Table 4: Stats when N varies.

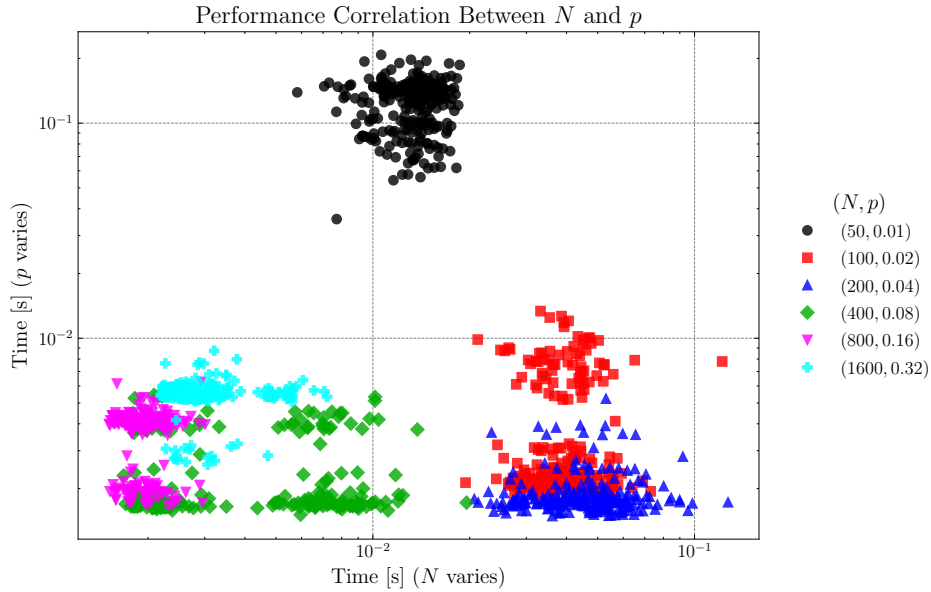
N	p		Time		Stopping index	
	mean	var	mean	var	mean	var
50	0.040000	0.000000	0.013228	0.000006	38.840000	20.464257
100	0.040000	0.000000	0.040066	0.000101	71.932000	55.188129
200	0.040000	0.000000	0.047342	0.000229	80.740000	97.253414
400	0.040000	0.000000	0.004659	0.000010	19.528000	28.226120
800	0.040000	0.000000	0.001961	0.000000	12.444000	0.263920
1600	0.040000	0.000000	0.003084	0.000001	10.572000	0.245799

After executing the experiments above, we obtain the plot in Figure 6.

Table 5: Stats when p varies.

p	N		Time		Stopping index	
	mean	var	mean	var	mean	var
0.010000	1000.000000	0.000000	0.124083	0.001042	116.880000	91.688353
0.020000	1000.000000	0.000000	0.003880	0.000008	17.072000	15.890378
0.040000	1000.000000	0.000000	0.001901	0.000000	11.764000	0.213157
0.080000	1000.000000	0.000000	0.002405	0.000001	10.000000	0.008032
0.160000	1000.000000	0.000000	0.003552	0.000001	8.868000	0.115036
0.320000	1000.000000	0.000000	0.005435	0.000001	8.000000	0.000000

Figure 6: Log-log plot of 200 iterations of the results in Table



2.2.2 Run with selective orthogonalization, no error bounds and $\delta = 10^{-5}$

3 Conclusions & future work

We didn't address the high memory cost of $O(N \cdot M)$ of Lanczos, this could be avoided by using a two-step implementation, that stores only three vectors and reconstructs the basis when needed. Further, because full orthogonalization was widely used in the second experiment, we could implement selective reorthogonalization as described in [4], but both methods fall behind the objective of this course. Another improvement, which we already discussed, would be to implement a working `WarpedTranslates` class in Library [1].

References

- [1] Michaël Defferrard, Lionel Martin, Rodrigo Pena, and Nathanaël Perraudin. Pygsp: Graph signal processing in python.
- [2] Ana Susnjara, Nathanael Perraudin, Daniel Kressner, and Pierre Vandergheynst. Accelerated filtering on graphs using lanczos method. 2015.
- [3] Ulrike Von Luxburg. A tutorial on spectral clustering. *Statistics and computing*, 17(4):395–416, 2007.
- [4] James W Demmel. *Applied numerical linear algebra*. SIAM, 1997.